Teste e Validação de Software

Segundo Projeto — Relatório

Dinis Rodrigues (ist106213) Artur Krystopchuk (ist104145)
Gabriel Santos (ist1116334)

Junho 2026

Resumo

Este relatório apresenta o desenho de casos de teste para o sistema de entregas urbanas descrito no enunciado do segundo projeto da disciplina Teste e Validação de Software. São aplicadas estratégias de desenho apropriadas para testar as classes Order e Delivery ao nível de classe, bem como os métodos `computeDeliveryCost()` e `updateItem()`. A última secção apresenta a concretização em TestNG de oito casos de teste seleccionados da bateria de testes da classe Order.

Conteúdo

1 Testes ao nível de classe da classe Order **2**
1.1 Estratégia escolhida: Teste de Classe Não-Modal . . . 2
1.2 Pontos On/Off . . . 2
1.3 Matriz de domínio com vetores de teste . . . 2
1.4 Invariante relacional(mesmo fornecedor) . . . 3

2 Testes ao nível de classe da classe Delivery **4**
2.1 Máquina de estados . . . 4
2.2 Transições válidas e inválidas . . . 4
2.3 Estratégia: *Round-Trip / Transition Coverage* . . . 5
2.4 Casos de teste . . . 5

3 Método `computeDeliveryCost()` **5**
3.1 Estratégia: Decision Table / Boundary Analysis . . . 5
3.2 Tabela de decisão . . . 5
3.3 Casos de teste . . . 5

4 Método `updateItem()` **5**
4.1 Estratégia escolhida . . . 5
4.2 Combinações estado + entrada . . . 5
4.3 Casos de teste . . . 5

5 Concretização em TestNG **5**

1 Testes ao nível de classe da classe Order

1.1 Estratégia escolhida: Teste de Classe Não-Modal

A classe Order não possui uma máquina de estados (não há modos que alterem o comportamento dos métodos); o seu comportamento ao nível de classe é determinado por um conjunto de *invariantes* sobre as suas variáveis de instância. A estratégia apropriada para testar uma classe deste tipo é o **Teste de Classe Não-Modal**, que consiste em: (i) identificar os domínios das variáveis; (ii) determinar os pontos *On*/*Off* de cada fronteira de invariante; e (iii) construir uma *matriz de domínio* com vetores de teste, em que cada caso de teste exercita uma fronteira de cada vez, mantendo as restantes variáveis em valores típicos.

1.2 Pontos On/Off

Fronteira (invariante)	**On**	**Off**
client $\neq$ `null`	Client válido	`null`
addr $\neq$ `null`	String válida	`null`
addrLength ≥ 1	1	0 (vazio)
distance ≥ 0	0	-1
distance ≤ 20000	20000	20001
quantity ≥ 1	1	0
quantity ≤ 10	10	11
cost ≤ 1000	1000	1000.01
totalQty $\leq 20 + \lfloor \text{cost}/50 \rfloor$	= limite	limite + 1

1.3 Matriz de domínio com vetores de teste

Cada coluna representa um caso de teste. Em cada coluna, exatamente uma fronteira está em *On* ou *Off* (célula sombreada), estando as restantes variáveis num valor típico (linha *In*). Valor típico adotado: encomenda com um item de preço 50 e quantidade 4 (cost = 200, totalQty = 4, limite = 24), morada de comprimento 3 e distance = 5000.

Tabela 2: Matriz de domínio para a classe Order.

Var.	**Condição**		**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
addr	≥ 1	On	1									
		Off		0								
	Típico	In			3	3	3	3	3	3	3	3
distance	≥ 0	On			0							
		Off				-1						
	≤ 20000	On					20000					
		Off						20001				
	Típico	In	5000	5000					5000	5000	5000	5000
quantity	≥ 1	On							1			
		Off								0		
	≤ 10	On									10	
		Off										11

Var.	Condição		1	2	3	4	5	6	7	8	9	10
	Típico	In	4	4	4	4	4	4				
cost	≤ 1000	On										
		Off										
	Típico	In	200	200	200	200	200	200	50	0	500	550
totalQty	$\leq$ lim.	On										
		Off										
	Típico	In	4	4	4	4	4	4	1	0	10	11
Resultado esperado			✓	✗	✓	✗	✓	✗	✓	✗	✓	✗

Tabela 3: Matriz de domínio para a classe Order (continuação: `cost`, `totalQty`, `client`, `addr` $\neq$ `null`).

Var.	Condição		11	12	13	14	15	16	17	18
client	$\neq$ `null`	On					ok			
		Off						`null`		
	Típico	In	ok	ok	ok	ok			ok	ok
addr	$\neq$ `null`	On							`"x"`	
		Off								`null`
	compr. típico	In	3	3	3	3	3	3		
distance	Típico	In	5000	5000	5000	5000	5000	5000	5000	5000
quantity	Típico	In	—	—	—	—	—	—	—	—
cost	≤ 1000	On	1000							
		Off		1000.01						
	Típico	In			200	200	200	200	200	200
totalQty	$\leq$ lim.	On			24					
		Off				25				
	Típico	In	20	20			4	4	4	4
Resultado esperado			✓	✗	✓	✗	✓	✗	✓	✗

Legenda: ✓ = estado válido (operação aceite); ✗ = estado inválido (deve lançar `InvalidOperationException` sem alterar o estado da encomenda, conforme I6).

1.4 Invariante relacional(mesmo fornecedor)

Este invariante não é numérico, pelo que não tem pontos On/Off. Testa-se com dois casos dedicados:

#	Cenário	Resultado esperado

19	Adicionar item de fornecedor S a encomenda já com itens do fornecedor S	Aceite (✓)
20	Adicionar item de fornecedor S' a encomenda já com itens do fornecedor S	`InvalidOperationException` (✗)

2 Testes ao nível de classe da classe **Delivery**

2.1 Máquina de estados

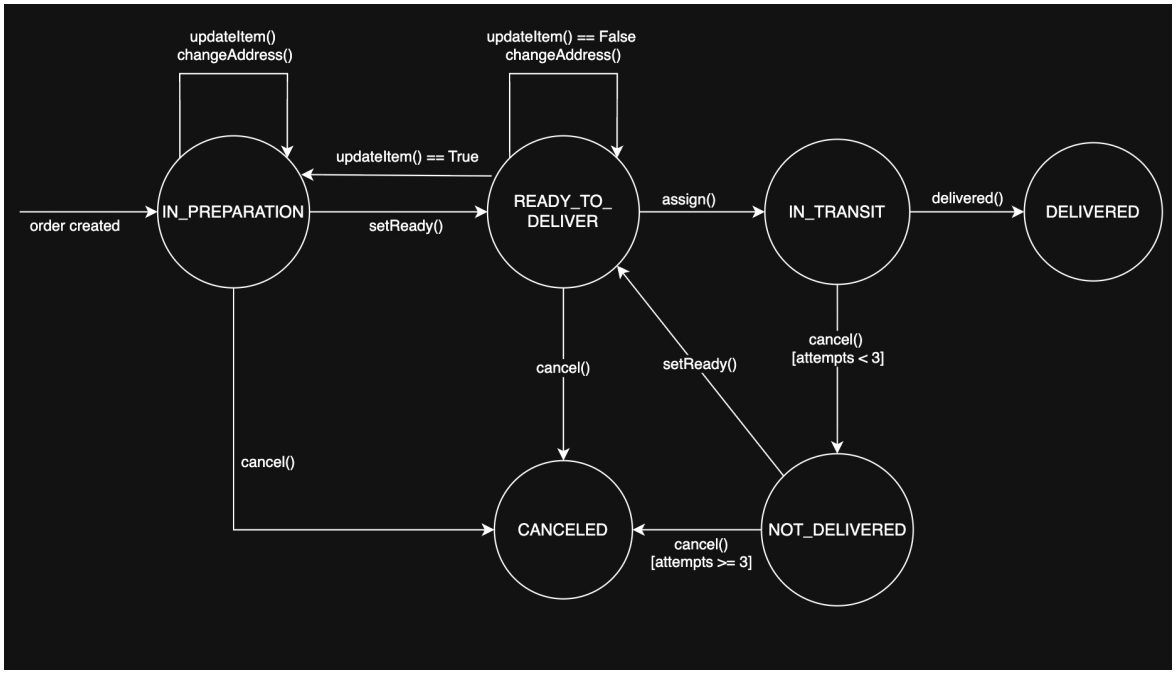


Figura 1: Máquina de estados da classe Delivery.

2.2 Transições válidas e inválidas

Tabela de transições a preencher.

2.3 Estratégia: *Round-Trip / Transition Coverage*

2.4 Casos de teste

3 Método `computeDeliveryCost()`

3.1 Estratégia: Decision Table / Boundary Analysis

3.2 Tabela de decisão

3.3 Casos de teste

4 Método `updateItem()`

4.1 Estratégia escolhida

4.2 Combinações estado + entrada

4.3 Casos de teste

5 Concretização em TestNG

Foram seleccionados 4 casos de sucesso e 4 casos de insucesso da bateria de testes da classe Order (Secção 1).

```java
package supereats.tests;

import org.testng.annotations.*;
import static org.testng.Assert.*;

import supereats.core.*;

public class OrderTest {

    private static final double EPS = 1e-9;

    private Supplier s1;   // fornecedor de referencia
    private Supplier s2;   // fornecedor distinto
    private Client   client;

    // Itens do fornecedor s1 (weight em gramas, price inteiro)
    private Item i1;       // preco 10
    private Item i2;       // preco 5
    private Item i50;      // preco 50  (para casos de totalQty)
    // Item do fornecedor distinto s2
    private Item iOther;

    @BeforeMethod
    public void setUp() {
        s1     = new Supplier("Loja A");
        s2     = new Supplier("Loja B");
        client = newClient();              // cliente valido qualquer

        i1     = new Item(100, 10, s1, "Item 1");
        i2     = new Item(200,  5, s1, "Item 2");
        i50    = new Item(150, 50, s1, "Item 50");
        iOther = new Item(100, 10, s2, "Item de outro fornecedor");
    }

    // Auxiliar: assume-se a existencia de um construtor de Client.
    private Client newClient() {
        return new Client(/* ... dados validos ... */);
```

```
38      }
39      // ---------- Casos de SUCESSO ----------
40
41      // TC1: construcao valida - todos os invariantes satisfeitos.
42      @Test
43      public void tc1_validConstruction() {
44          Order o = new Order(client, "Rua A", 5000);
45          assertEquals(o.totalQuantity(), 0);
46          assertEquals(o.cost(), 0.0, EPS);
47          assertTrue(o.itens().isEmpty());
48      }
49
50      @Test
51      public void T9_addItem_emptyOrder_validQuantity() {
52          Order o = new Order(client, "Rua A", 5000);
53          o.addItem(i1, 1);
54          assertTrue(o.contains(i1));
55          assertEquals(o.quantity(i1), 1);
56          assertEquals(o.cost(), 10.0, EPS);
57      }
58
59
60      // TC13: fronteira totalQty <= 20 + floor(cost/50) (On).
61      // Usa-se preco unitario 1 para isolar a fronteira de totalQty sem
62      // violar o limite de custo (I3). Com cost = 24 * 1 = 24, o limite e'
63      // 20 + floor(24/50) = 20 + 0 = 20... pelo que e' preciso elevar o
64      // custo o suficiente para que o limite atinja 24:
65      //   limite = 24  <=>  floor(cost/50) = 4  <=>  cost em [200, 250).
66      // Escolhe-se cost = 240 com itens de preco 10 (24 unidades * 10 = 240),
67      // mantendo cost <= 1000.
68      @Test
69      public void tc13_totalQuantityAtDynamicLimit() {
70          Order o = new Order(client, "Rua A", 5000);
71          // 3 itens de preco 10, quantidades 10 + 10 + 4 = 24 unidades
72          Item a = new Item(150, 10, s1, "A");
73          Item b = new Item(150, 10, s1, "B");
74          Item c = new Item(150, 10, s1, "C");
75          o.addItem(a, 10);
76          o.addItem(b, 10);
77          o.addItem(c, 4);
78          assertEquals(o.totalQuantity(), 24);   // = 20 + floor(240/50) = 24
79          assertEquals(o.cost(), 240.0, EPS);
80      }
81
82      // TC19: - adicionar item do MESMO fornecedor é aceite.
83      @Test
84      public void tc19_sameSupplier_isAccepted() {
85          Order o = new Order(client, "Rua A", 5000);
86          o.addItem(i1, 2);
87          o.addItem(i2, 3);                      // i2 tambem do fornecedor s1
88          assertTrue(o.contains(i1));
89          assertTrue(o.contains(i2));
90          assertEquals(o.totalQuantity(), 5);
91          assertEquals(o.cost(), 2*10 + 3*5, EPS);
92      }
93
94      // ---------- Casos de INSUCESSO ----------
95
96      // TC2: fronteira addrLength >= 1 (Off) - morada vazia.
97      @Test(expectedExceptions = InvalidOperationException.class)
98      public void tc2_emptyAddress_throws() {
99          new Order(client, "", 5000);
100     }
```

```
101
102     // TC6: cliente null.
103     @Test(expectedExceptions = InvalidOperationException.class)
104     public void tc6_nullClient_throws() {
105         new Order(null, "Rua A", 5000);
106     }
107
108     // TC10: fronteira quantity <= 10 (Off) - quantidade 11.
109     // Verifica tambem que o estado nao e' alterado (I6).
110     @Test
111     public void tc10_quantityAboveMax_throwsAndKeepsState() {
112         Order o = new Order(client, "Rua A", 5000);
113         try {
114             o.addItem(i1, 11);
115             fail("Esperava InvalidOperationException");
116         } catch (InvalidOperationException e) {
117             assertFalse(o.contains(i1));       // estado inalterado
118             assertEquals(o.totalQuantity(), 0);
119         }
120     }
121
122     // TC20: adicionar item de fornecedor DISTINTO.
123     @Test(expectedExceptions = InvalidOperationException.class)
124     public void tc20_differentSupplier_throws() {
125         Order o = new Order(client, "Rua A", 5000);
126         o.addItem(i1, 1);                     // fornecedor s1
127         o.addItem(iOther, 1);                 // fornecedor s2 -> excecao
128     }
129 }
```

Listing 1: Esqueleto da classe de testes TestNG para Order.